

Навигатор по claude-code-best- practice

Топ-советы по Claude Code,
разложенные по реальной пользе

Что ставить сегодня, что ситуативно, а что мимо

Игорь Масленников · AI Dev Team Июнь 2026 · версия 1.0

Это спутник к статье [«Топ-советы по Claude Code от Бориса Черни и не только»](#). Оригинальный репозиторий — [shanraishan/claude-code-best-practice](#) (под 56 тысяч звёзд на момент написания, июнь 2026). Там собрано всё; здесь — только то, что я считаю стоящим внимания, и разложено по приоритету пользы.

Зачем этот навигатор

Репозиторий `claude-code-best-practice` — отличная карта: 83 совета, разбор примитивов, 13 методологий, cross-model подходы. Но у карты есть две проблемы. Первая: советы лежат единым списком, без приоритета — новичку непонятно, с чего начинать. Вторая, и более глубокая: это **агрегатор опыта, которого пока ни у кого толком нет**. Claude Code, агентной разработке, скиллам — всему от силы год, модели обновляются каждые шесть недель, и «лучшая практика» живёт месяца три, прежде чем половина советов протухает.

Поэтому навигатор — не «правильные ответы», а **прожитая приоритизация на июнь 2026**. Я прошёлся по советам и инструментам и разложил их по группам: что я считаю обязательным, что просто полезным, что на любителя, что переоценено и что honestly мимо. Где у меня есть личный вердикт из практики — он помечен отдельно.

Через квартал часть оценок сдвинется. Это нормально. В области, где чужой опыт устаревает за квартал, дороже всего собственные замеры и обсуждение — за тем и собираю это в живой документ, который буду обновлять.

Как устроены приоритеты

Пять групп, от «ставь сегодня» до «honestly мимо»:

- **★★★ Ультраполезное** — то, что меняет работу с агентами в первый же день. Если возьмёте из навигатора только это — уже окупится.
- **★★ Полезное** — твёрдые практики, которые стоит внедрить, когда дошли руки.
- **★ Ситуативное** — работает не у всех и не всегда; зависит от задачи и вкуса.
- **▲ Переоценено / осторожно** — совет популярен, но у него есть скрытая цена или его легко применить во вред.
- **× Не пробовал / мимо** — либо я честно не проверял, либо не почувствовал необходимости.

Метки: - **[мой вердикт]** — оценка из собственной практики, не пересказ. - **[черновик]** — пока предположительная раскладка, могу переставить в следующей версии. - **→ статья** — у меня есть отдельный разбор по теме, ссылка в конце.

★★★ Ультраполезное

Дать модели способ проверить собственный результат. Главный совет всей подборки, и он от самого создателя Claude Code. Как только у агента есть способ убедиться, что работа сделана (тесты, линтер, прогон), он будет итерировать, пока не станет хорошо. Без этого автономия превращается в «агент сказал, что готово». **[мой вердикт]** — золото, у меня вокруг этой мысли построена вся работа с агентами. → статья про честность Opus 4.8.

Начинать в plan mode. Shift+Tab дважды, согнать план туда-сюда, согласиться — и только потом пускать в авто-режим. Хороший план важнее быстрого старта. **[мой вердикт]** — золото, почти каждая нетривиальная задача у меня так и идёт.

Просить не один результат, а 3-4 варианта. Стоимость генерации низкая, поэтому дешевле посмотреть несколько живых вариантов и выбрать лучший, чем расписывать словами, чего хочешь. Особенно на UI/UX. **[мой вердикт]** — золото, использую постоянно.

Держать в голове гниение контекста (~40%). Модель умнеет не до конца окна — качество начинает плыть гораздо раньше, на практике где-то к 40% заполнения. Не доводите длинную сессию до потолка. **[мой вердикт]** — стопроцентно мой опыт. Ловушка: в Claude Code с 1M контекста за этим приходится следить руками (в Codex/ChatGPT окно меньше, зато компактится само). → статья про Opus 4.8.

Новая задача — новая сессия. Не тащить пять разных задач в один контекст. Дешёвая гигиена, экономит и токены, и качество. **[черновик]**

Откатывать неудачную попытку, а не дописывать поверх. /rewind лучше, чем «исправить» — провальная попытка засоряет контекст и тянет качество вниз. **[черновик]**

★★ Полезное

Субагенты для изоляции контекста. Тяжёлый поиск или чтение — в отдельного субагента с собственным окном; основной контекст остаётся чистым.

Команды (`/commands`) для повторяющихся процессов. Если делаешь что-то чаще раза в день — оформи командой, держи в git. Надёжнее, чем каждый раз промптить заново.

Скиллы для повторяемых задач. Папка со `SKILL.md` , скриптами и примерами. Описание скилла должно запускать поведение, а не пересказывать очевидное.

Verification-скиллы окупаются. Скиллы, которые прогоняют продукт по сценарию (signup-flow, checkout) ловят то, что юнит-тесты не видят.

CLAUDE.md держать коротким. Цель — меньше 200 строк (а лучше 60). Большие правила раскладывать по `.claude/rules/*.md` с ленивой загрузкой. Длинное полотно правил агент игнорирует.

settings.json вместо прозы. Детерминированное поведение лучше задавать настройками, чем уговаривать словами в CLAUDE.md.

PostToolUse hook для автоформатирования. Пусть код причёсывается сам после правок, а не «не забудь отформатировать».

Squash merge и мелкие частые PR. Чистая линейная история, лёгкий revert/bisect. У Черни медиана — 118 строк на PR, всё squash. **[мой вердикт]** — с этим спорить трудно.

`/compact` с подсказкой лучше авто-compact. Подскажи, что сохранить, — компактизация будет осмысленнее. `/clear` — для high-stakes, когда нужен чистый лист.

ASCII-диаграммы для архитектуры. Простой способ заставить модель и себя увидеть структуру до того, как писать код.

Cross-model для ревью. Прогнать чужой моделью (например, Codex ревьюит то, что сделал Claude) — ловит то, что родная модель не замечает. → дайджест по Codex.

Скриншоты при застревании + MCP для консоли. Когда агент буксует на UI — дать ему глаза (Chrome/Playwright MCP, скриншот).

★ Ситуативное

Prototype > PRD (построй 20-30 версий вместо ТЗ). Мощно для UI/UX — там и правда дешевле настрогать вариантов. Опасно для архитектуры, где 20 прототипов без плана дают 20 тупиков. **[мой вердикт]** — для UI золото (см. ★★★), для системного дизайна осторожно.

Agent Teams (tmux + worktrees). Координация нескольких Claude для параллельной разработки. Дорого (×3-7 от обычной сессии). Под узкий класс задач. → дайджест по Claude.

Долгие автономные циклы (/loop , /schedule , Ralph Wiggum). Для длинных фоновых задач. Заходит не всем и требует хорошей верификации, иначе агент долго и уверенно делает не то.

Voice dictation (/voice , Wispr Flow). Кому-то кратно ускоряет ввод, кому-то мешает. Чистая вкусовщина.

Терминал вместо IDE (iTerm/Ghostty/tmux). Дело привычки. Кто живёт в терминале — выигрывает, остальным не обязательно.

▲ Переоценено / осторожно

«**Не нянчи Claude**». Верно про микроменеджмент шагов — не надо диктовать, как именно чинить баг. Но без способа проверить результат это превращается в слепое доверие. Completion event $\neq$ acceptance. **[мой вердикт]** — совет полезен ровно в паре с «дай модели проверить себя» из ★★★. → статья про Orus 4.8.

1M контекста как «решение». Большое окно продаётся как преимущество, а на практике это ещё и налог: за заполнением приходится следить руками, иначе словишь гниение. **[мой вердикт]** — не достоинство, а цена, про которую в восторженных тредах забывают.

Векторный RAG для навигации по коду. Авторы Claude Code сами его попробовали и выкинули — для поиска по коду точность/свежесть/простота у grep выше. НО: это не значит «индексация не нужна». Для кода выигрывает гибрид grep + структурный индекс (tree-sitter/AST), а не чистый вектор и не чистый grep. **[мой вердикт]** — переоценён именно векторный RAG, а не индексация как таковая. → статья «CodeGraph vs Graphify».

× Не пробовал / мимо

`/sandbox` (**−84% запросов на подтверждение**). Запуск действий агента в OS-песочнице, чтобы не подтверждать каждую команду. Хвалят. **[мой вердикт]** — не пробовал, необходимости не почувствовал. Возможно, дозрею — но пока честно мимо.

Этот раздел будет расти по мере того, как я прогоню остальные советы. Если у вас есть кандидаты в «мимо» из собственного опыта — присылайте, добавлю с указанием источника.

Карта 13 методологий: когда какую брать

В репозитории собраны 13+ методологий оркестрации. При всей разности они сходятся к одному скелету: **Research → Plan → Execute → Review → Ship**. Так что выбирать стоит не «правильную», а удобную под ваш стиль:

Методология	Когда брать
Superpowers	Нужны процессные скиллы (brainstorm → worktrees → план → subagent-driven dev). Мой основной выбор.
Спец Kit	Любите спеки и формальные гейты: constitution → specify → implement.
gstack	Командный процесс с ревью-сессиями («office hours»).
HumanLayer	Упор на handoff между человеком и агентом, research-heavy задачи.
BMAD-METHOD	Продуктовый цикл PRD → PRFAQ → архитектура → истории.
OpenSpec	Лёгкий propose → apply → verify, если Спец Kit кажется тяжёлым.
Get Shit Done / agent-skills / Compound Engineering / oh-my-claudecode	Вариации того же скелета — берите ту, чьи команды и тон вам ближе.

Не пытайтесь освоить все 13. Возьмите одну, проживите месяц, потом сравните.

Мои разборы по узлам (deep-dive)

Навигатор показывает «что и куда». Подробности — в отдельных статьях:

- **Оркестрация через контракты** — как собрать систему из скиллов, Parallel Decomposition Matrix, worker-контракты.
- **Автономия и честность агентов** — почему сотня субагентов бесполезна, если они врут; completion $\neq$ acceptance.
- **Индексация кода** — CodeGraph vs Graphify, почему индекс это регенерируемый кэш, а не коммитимый артефакт.
- **Память агентов** — разбор gbrain: три слоя памяти, гибридный поиск.
- **Выбор модели под продакшен** — боевой тест на русском контенте.

(Ссылки — в статье-спутнике и в Telegram-канале.)

Контакты и обратная связь

Навигатор составлен **Игорем Масленниковым** — руковожу AI Dev Team, ежедневно работаю с агентскими системами на Claude Code и Codex.

Telegram-канал: t.me/maslennikovigor Наблюдения по работе с агентами, разборы инструментов, обновления этого навигатора.

Прямой контакт: [@maslennikovig](https://t.me/maslennikovig) Не согласны с приоритетом? Прогнали совет и получили другой результат? Напишите — это ровно тот документ, который должен спорить и обновляться.

Инструменты оркестрации: [maslennikov-ig/claude-code-orchestrator-kit](https://github.com/maslennikov-ig/claude-code-orchestrator-kit)

Дисклеймер

Версия 1.0 на июнь 2026. Часть оценок помечена как предположительная ([черновик]) — буду уточнять по мере того, как прогоню оставшиеся советы сам. Оригинальный репозиторий и точные формулировки советов — у автора, [shanraishan/claude-code-best-practice](https://github.com/shanraishan/claude-code-best-practice). Числа (количество звёзд, «—84% промптов», «118 строк на PR», «гниение на 40%») — на момент составления, по заявкам репозитория и его источников; перед тем как опираться на них, сверяйтесь с первоисточником.

Не маркетинговый материал, не платная реклама. Распространяется бесплатно — можно пересылать и цитировать с указанием источника.

Версия 1.0 · Июнь 2026